SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for

recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:25**

Duration : 1 Hour

AT1- Application Based Question

Code of Conduct:

I **J. SAI DHEERAJ (192311191)** (Name / Reg No) certify that this submission is my original work and that I have

adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J. SAI DHEERAJ

SIMATS ENGINEERING

ASSESSMENT TOOL

1.In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a Deterministic Finite Automaton (DFA) over the alphabet Σ = {0,1} to validate passwords that end with the pattern "01".Further, minimize the DFA and analyze how the optimized design improves system efficiency in real-world hardware implementations

Aim

To design a Deterministic Finite Automaton (DFA) over the alphabet $\Sigma=\{0,1\}$ that accepts all binary strings ending with 01 and minimize the DFA.

DFA Design

The DFA must accept a string only when its last two symbols are 01.

Define the DFA as:

$$ M=(Q,\Sigma,\delta,q_0,F) $$

Where:

- $Q=\{q_0,q_1,q_2\}$ is the set of states.
- $\Sigma=\{0,1\}$ is the input alphabet.
- q_0 is the initial state.
- $F=\{q_2\}$ is the set of final states.
- δ is the transition function.

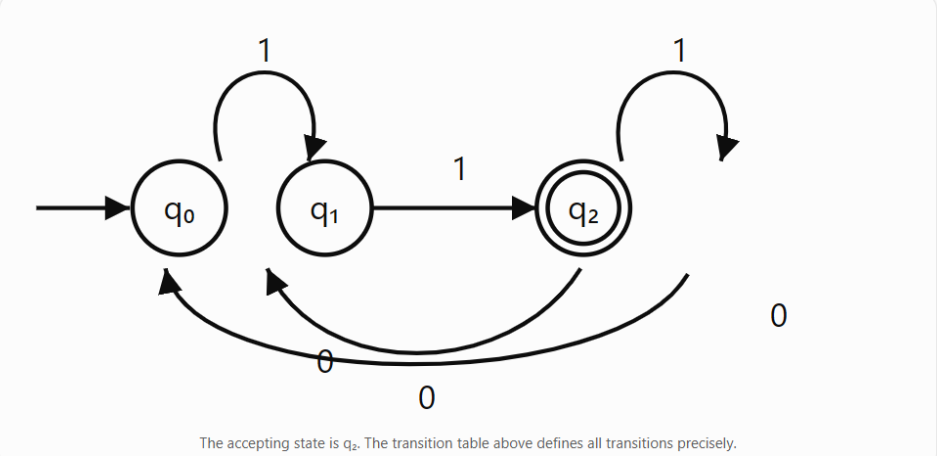
State transition table

Current State	Input 0	Input 1
→ q_0	q_1	q_0
q_1	q_1	q_2

*\(q_2\) \(q_1\) \(q_0\)

Here, → indicates the initial state and * indicates the accepting state.

State diagram



The accepting state is q_2. The transition table above defines all transitions precisely.

Working

- \(q_0\): The string does not currently end in 0 or 01.
- \(q_1\): The last symbol read is 0.
- \(q_2\): The last two symbols are 01; hence, the string is accepted.

Examples:

Input	Result	Reason
101	Accepted	Ends with 01
1101	Accepted	Ends with 01
010	Rejected	Ends with 10
111	Rejected	Does not end with 01

DFA Minimization

The three states are distinguishable:

- \(q_0\) and \(q_1\) differ because the suffix 1 is accepted from \(q_1\) but not from \(q_0\).
- \(q_0\) and \(q_2\) differ because the empty string is accepted from \(q_2\) but not from \(q_0\).
- \(q_1\) and \(q_2\) differ because the empty string is accepted from \(q_2\) but not from \(q_1\).

Therefore, the minimized DFA contains three states, and no further reduction is possible.

Real-world applications

- Reduces unnecessary state transitions and memory usage compared with a larger, nonminimal design.
- Improves hardware implementation by reducing the number of state registers and associated logic when a larger design can be minimized.
- Supports predictable, constant-time processing per input symbol.
- Can be used in embedded security devices and binary protocol validation.

Conclusion: A three-state DFA efficiently recognizes all binary strings ending with 01.

2.In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet Σ = {a, b}.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

Aim

To construct a regular expression that accepts exactly the string aaaabb over the alphabet $\Sigma=\{a,b\}$.

Regular Expression

$$\boxed{R=aaaabb}$$

The expression contains exactly four occurrences of a, followed by two occurrences of b.

An equivalent notation using repetition is:

$$\boxed{R=a^4b^2}$$

Validation examples

Input	Result
aaaabb	Accepted

aaaab Rejected

aaaaabb Rejected

aaaabbb Rejected

aabb Rejected

For a web application using JavaScript, the corresponding regular expression is:

^aaaabb$

The ^ symbol specifies the beginning of the input, and $ specifies the end. Therefore, additional characters before or after the required pattern are not allowed.

Use of regular languages in web applications

Regular expressions provide a concise method for validating structured inputs.

Applications include:

1. Form validation: Checking usernames, postal codes, and fixed-format identifiers.
2. Security: Rejecting inputs that do not follow an expected format before processing them.
3. Data consistency: Ensuring that submitted values conform to predefined rules.
4. Performance: Matching a fixed pattern can be performed efficiently without requiring complex parsing.

Regular expressions should complement, rather than replace, server-side validation and security controls. For example, checking an email's format does not prove that the email address exists.

Conclusion: The regular expression ^aaaabb$ accepts only the exact required string, making it suitable for fixed-pattern validation.

3.In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a*b* can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

Aim

To convert the regular expression a^*b^* into an equivalent finite automaton and explain its use in real-time pattern detection.

Meaning of the expression

$$\boxed{a^*b^*}$$

Here:

- a^* represents zero or more occurrences of a.
- b^* represents zero or more occurrences of b.

Therefore, the language contains any number of a symbols followed by any number of b symbols. Either group may be empty.

Examples of accepted strings are:

$$\epsilon,\ a,\ aa,\ b,\ bb,\ ab,\ aabb,\ aaabbb$$

The string ba is rejected because an a occurs after a b.

Finite automaton design

Use the following DFA:

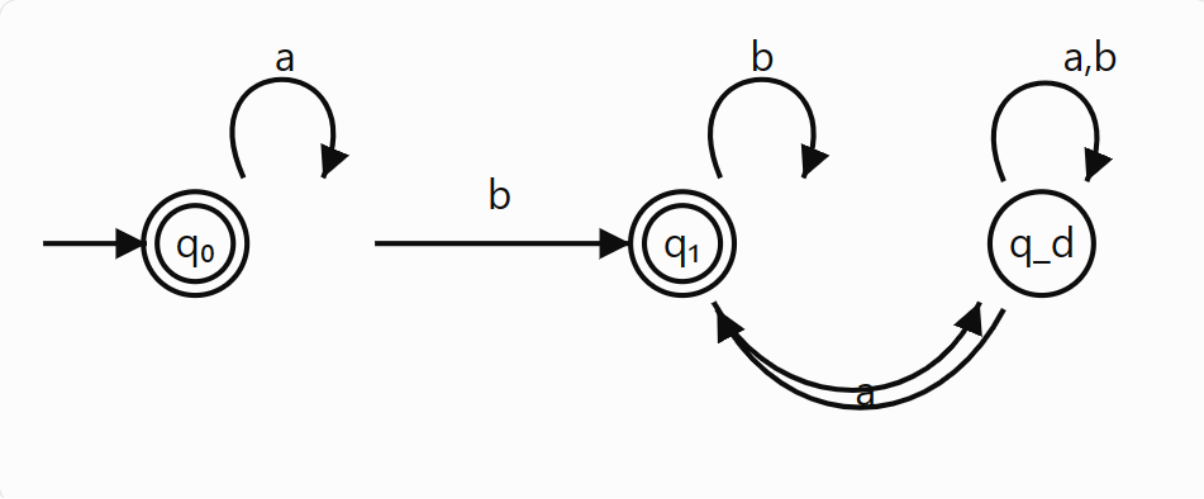
- q_0: Reads the initial sequence of a symbols.
- q_1: Reads the sequence of b symbols.
- q_d: Dead state entered if an a occurs after a b.

The initial state is q_0, and the accepting states are q_0 and q_1.

Transition table

State	Input a	Input b
→ *q_0	q_0	q_1
*q_1	q_d	q_1
q_d	q_d	q_d

State diagram



Why regular expressions and finite automata are equivalent

Regular expressions and finite automata describe exactly the class of regular languages.

- Every regular expression can be converted into an equivalent finite automaton.
- Every finite automaton can be represented by an equivalent regular expression.
- Conversions can be performed using methods such as Thompson's construction and state elimination.

Real-time applications

- Intrusion detection: Recognizes suspicious patterns in network traffic.
- Compiler design: Identifies tokens such as keywords, operators, and identifiers.
- Log monitoring: Detects predefined event sequences.
- Input validation: Checks whether data follows an allowed pattern.

Finite automata process symbols incrementally, requiring no unbounded memory for regular-language recognition. This makes them suitable for high-speed pattern matching.

Conclusion: The regular expression \(a^*b^*\) is recognized by a finite automaton that accepts all a-then-b strings and rejects strings containing an a after a b.

4.In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language L= { anbn | n ≥ 0 } cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

Aim

To use the pumping lemma to prove that the language $L=\{a^nb^n\mid n\geq0\}$ cannot be recognized by any finite automaton.

Pumping Lemma

The pumping lemma states that if a language L is regular, there exists a positive integer p, called the pumping length, such that every string $w\in L$ with $|w|\geq p$ can be written as:

$$ w=xyz $$

such that:

1. $|xy|\leq p$.
2. $|y|>0$.
3. $xy^iz\in L$ for every integer $i\geq0$.

Proof by contradiction

Step 1: Assume the language is regular.

Suppose $L=\{a^nb^n\mid n\geq0\}$ is regular. By the pumping lemma, it has a pumping length p.

Step 2: Choose a suitable string.

Consider the string:

$$ w=a^pb^p $$

This string belongs to L because it contains exactly p a symbols followed by exactly p b symbols.

Step 3: Divide the string.

Since $|xy|\leq p$, both x and y must lie entirely within the first p a symbols.

Thus:

$$ y=a^k,\qquad k\geq1 $$

Step 4: Pump down.

Choose $i=0$. The resulting string is:

$$ xz=a^{p-k}b^p $$

This string contains fewer a symbols than b symbols. Therefore, it does not belong to L.

Step 5: Reach a contradiction.

The pumping lemma requires $xz\in L$, but we have shown that $xz\notin L$.

Hence, the original assumption is false.

$$\boxed{L\text{ is not a regular language.}}$$

Implications for modern compiler design

Finite automata cannot recognize every possible programming-language structure.

- Lexical analysis: Finite automata are suitable for tokens such as identifiers, numbers, and operators.
- Syntax analysis: Context-free grammars and pushdown automata are needed for nested structures and balanced parentheses.
- Semantic analysis: Additional checks verify types, declarations, and other program constraints.
- Memory limitations: A finite automaton has only a fixed number of states and cannot count an arbitrary number of a symbols and compare that count with the number of b symbols.

For example, recognizing arbitrarily nested parentheses requires a stack or another mechanism with unbounded memory.

Conclusion: The pumping lemma proves that $L=\{a^nb^n\mid n\geq0\}$ is not regular. Modern compilers therefore combine finite automata with more powerful parsing techniques.

5.Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the closure properties of regular languages enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations remains regular, ensuring efficient implementation using finite automata

Aim

To explain how closure properties allow multiple pattern-matching rules to be combined while preserving regularity.

Meaning of closure properties

A language family is said to be closed under an operation if applying that operation to languages in the family always produces another language in the same family.

Regular languages are closed under several important operations.

Major closure properties

Operation	Meaning	Example
Union	Accept strings belonging to either language	$L_1\cup L_2$
Intersection	Accept strings belonging to both languages	$L_1\cap L_2$
Complement	Accept strings not belonging to a language	$\overline{L_1}$
Difference	Accept strings in one language but not another	L_1-L_2
Concatenation	Join a string from each language	L_1L_2
Kleene star	Repeat strings zero or more times	L_1^*

Example: Combining detection rules

Suppose a security system has two regular languages:

- L_1: Strings matching the pattern ab.
- L_2: Strings matching the pattern ba.

Their union is:

$$ L=L_1\cup L_2 $$

The combined language accepts either ab or ba.

The union can be implemented by constructing an automaton that accepts either pattern. Likewise, intersection can be implemented using a product automaton, and complement can be implemented by making a complete DFA and swapping its accepting and nonaccepting states.

Applications in real-world systems

1. Intrusion detection systems

Multiple regular-expression rules can be combined to recognize different suspicious traffic patterns. A union allows the system to flag traffic matching any one of several patterns.

2. Spam filtering

Rules that identify suspicious phrases, repeated symbols, or predefined message formats can be combined to detect potential spam. Such rules are indicators, not proof that a message is malicious.

3. Web input validation

Applications can combine rules for allowed characters, prefixes, and fixed formats. Intersection can represent inputs that must satisfy multiple regular-language constraints.

4. Compiler construction

Regular-language rules can be combined to recognize different token types, including identifiers, numeric literals, and operators.

Why does the resulting language remain regular?

The closure properties guarantee that applying any of the listed operations to regular languages produces another regular language. Therefore, an equivalent finite automaton exists for the resulting language.

Finite automata can process each input symbol using a finite transition table. This supports efficient recognition and makes the approach practical for many real-time detection tasks.

However, combining many rules may increase the number of states in the resulting automaton. Practical implementations may need optimization to control memory usage and processing time.

Conclusion: Closure properties make regular languages flexible and composable. They allow multiple detection and validation rules to be combined while preserving the ability to recognize the resulting language using finite automata.